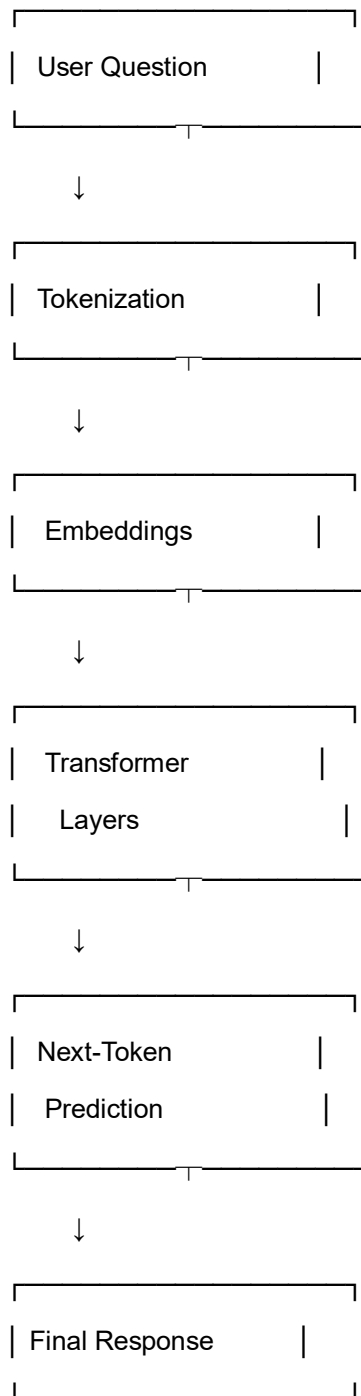


# What Happens When You Ask an LLM a Question?

When you ask a **Large Language Model (LLM)** a question, the model does not simply look up a pre-written answer. Instead, it processes your input through several computational stages and generates a response based on patterns learned during training.

At a high level, the process looks like this:



In simple terms:

**Question → Tokens → Numerical Representations → Transformer → Token Prediction → Answer**

## 1. The User Sends a Prompt

The process begins when the user enters a **prompt**. A prompt can be a question, instruction, code snippet, document, or combination of different types of information.

For example:

"Why is the sky blue?"

Depending on the AI application, the model may receive more than just the visible question. Its input context can also contain:

- Previous conversation messages
- System or application instructions
- Retrieved documents
- Search results
- Tool outputs
- Other relevant contextual information

All of this information forms the **context** that the model uses to generate its response.

## 2. The Prompt Is Converted Into Tokens

LLMs cannot directly process text in the same way humans read sentences. The input is first divided into smaller units called **tokens**.

A token may represent a complete word, part of a word, punctuation mark, number, or symbol.

For example:

"Artificial intelligence is powerful."

↓

Artificial | intelligence | is | powerful | .

Each token is mapped to a numerical **token ID**. These IDs allow the model to process language computationally.

The exact tokenization depends on the tokenizer used by the particular model.

## 3. Tokens Become Embeddings

Token IDs alone do not represent the meaning or relationships between words. Therefore, each token is converted into an **embedding**.

An embedding is a high-dimensional vector of numbers that provides the neural network with a mathematical representation of the token.

Conceptually:

"computer"



Token ID



[0.18, -0.42, 0.73, 0.21, ...]



Embedding

The model also needs information about the **position** of each token. This matters because word order can completely change meaning.

For example:

"The dog chased the cat."

and

"The cat chased the dog."

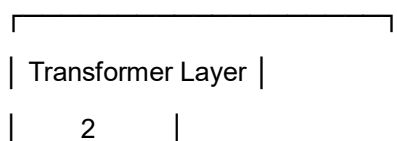
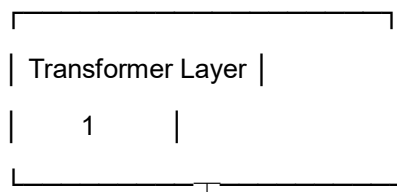
contain almost identical words but describe different events. Positional information helps the model distinguish between these sequences.

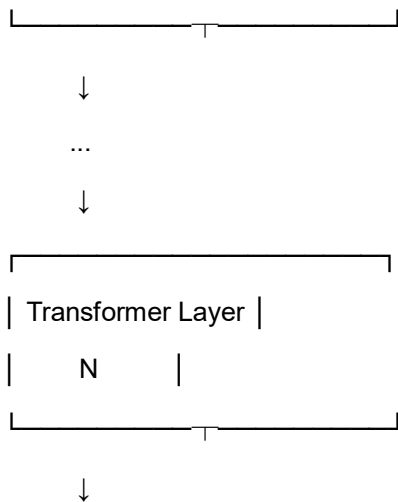
#### 4. The Transformer Processes the Context

The numerical representations are then passed through the **Transformer**, the neural-network architecture behind most modern LLMs.

A Transformer contains multiple layers that progressively process and refine the representation of the input.

Input Embeddings





Processed Context

One of the most important mechanisms inside these layers is **self-attention**.

### Self-Attention

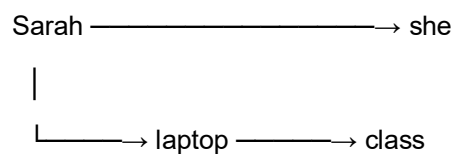
Self-attention helps the model determine which parts of the context are relevant to one another.

Consider:

"Sarah picked up her laptop because she needed it for class."

When processing this sentence, attention mechanisms can help connect concepts such as **Sarah**, **laptop**, **she**, and **class**.

A simplified representation is:



This ability to model relationships across the context is one reason Transformers are effective at processing language.

## 5. The Model Predicts the Next Token

After processing the context, the model produces scores for possible **next tokens**.

For example, suppose the input is:

"The capital of France is..."

The model might assign different probabilities to possible continuations:

### Possible Next Token Example Probability

|       |     |
|-------|-----|
| Paris | 92% |
| Lyon  | 2%  |

### Possible Next Token Example Probability

|              |      |
|--------------|------|
| London       | 0.4% |
| Rome         | 0.2% |
| Other tokens | 5.4% |

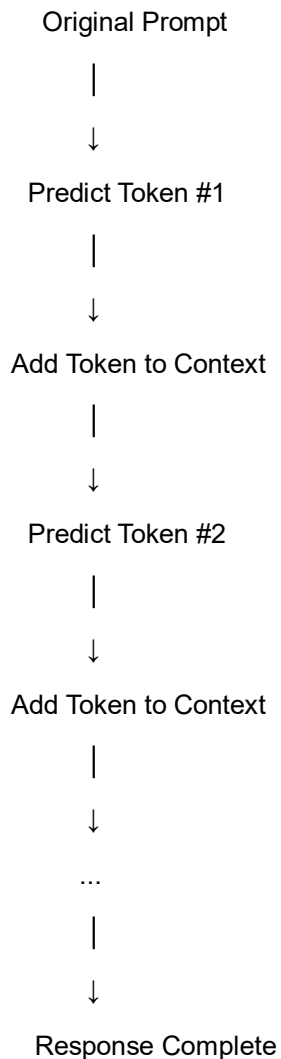
These probabilities are illustrative, not actual model values.

The system then selects a token using a **decoding strategy**. Depending on the configuration, generation can favor the highest-probability token or allow some controlled variation.

### 6. The Prediction Process Repeats

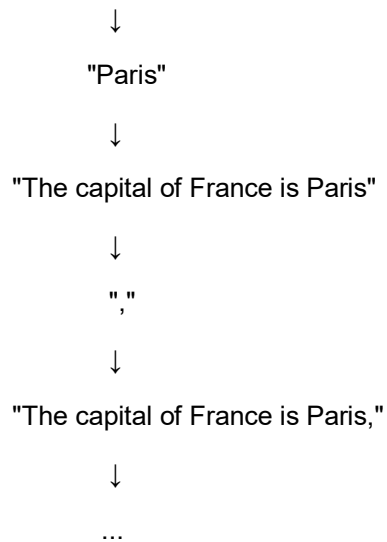
The LLM does not normally generate the entire answer in one operation. Instead, it generates the response **token by token**.

After predicting one token, that token becomes part of the context used to predict the next token.



For example:

"The capital of France is"

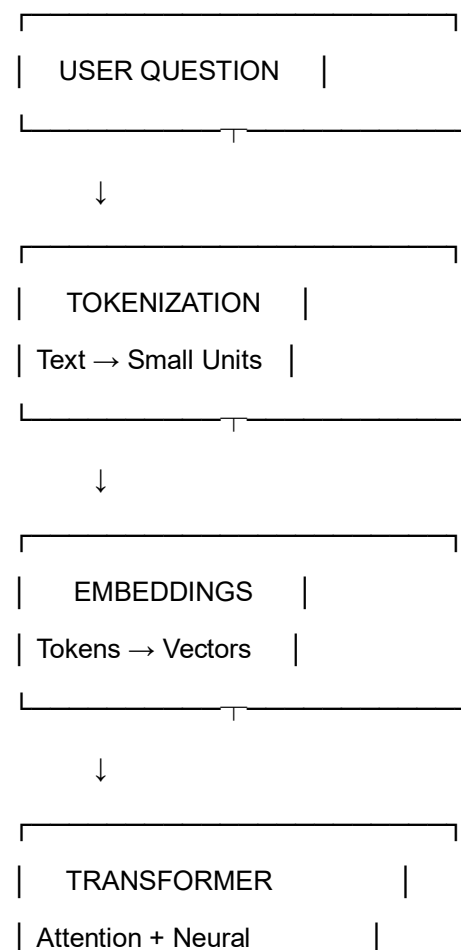


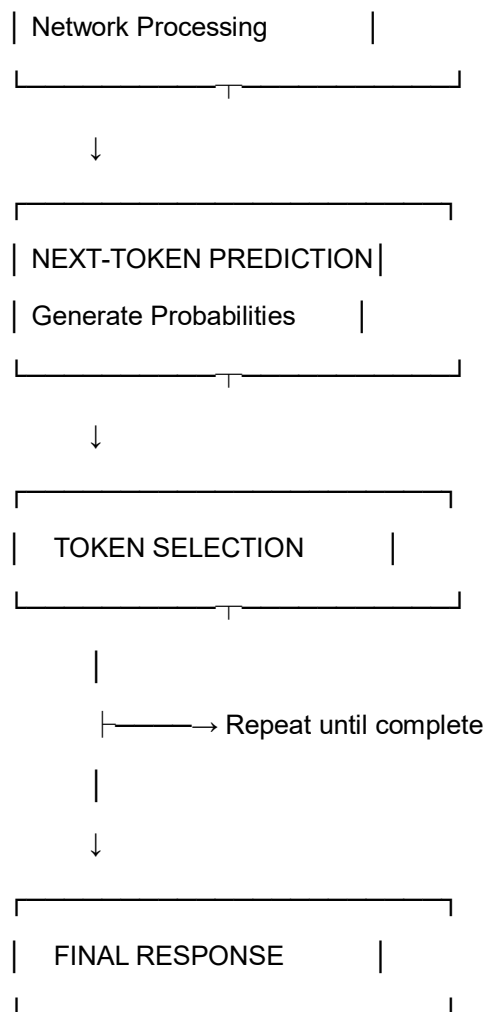
This process is called **autoregressive generation**.

## 7. The Final Response Is Produced

Once the generation process reaches a stopping condition, the generated tokens are converted back into readable text and presented to the user.

The complete process can therefore be summarized as:



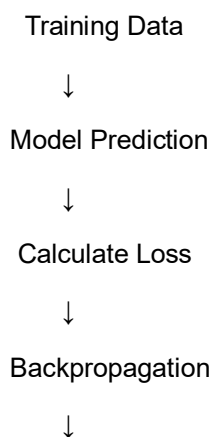


### Where Does an LLM's Knowledge Come From?

Before an LLM can answer questions, it must go through **training**. During training, the model processes large amounts of data and repeatedly attempts to predict tokens.

When its prediction differs from the expected continuation, an error value called a **loss** is calculated. Training algorithms then adjust the model's numerical parameters, or **weights**, to reduce this error.

A simplified training process is:



Update Weights



Repeat



Trained Model

After many training iterations, the model's parameters encode statistical patterns involving language, concepts, relationships, and other structures found in its training data.

## Training vs. Inference

It is important to distinguish **training** from **inference**.

### Training

Training is the process in which the model learns by adjusting its parameters.

**Training Data → Prediction → Loss → Update Weights → Repeat**

### Inference

Inference happens when a trained model receives a new prompt and generates a response.

**User Prompt → Trained Model → Token Predictions → Response**

During normal inference, the base model generally uses its existing learned parameters rather than retraining itself for every question.

## Why Can an LLM Give Wrong Answers?

An LLM predicts plausible continuations based on patterns learned during training. A statistically plausible answer is not necessarily a factually correct answer.

Incorrect responses can occur because:

- The model's information may be incomplete or outdated.
- The question may be ambiguous.
- The supplied context may contain incorrect information.
- The model may incorrectly combine learned patterns.
- The question may require information unavailable to the model.
- A plausible but incorrect token sequence may be generated.

A confident-sounding but incorrect or unsupported response is commonly called a **hallucination**.

For this reason, important factual information generated by an LLM should be verified using reliable sources.



## Key Terms

- **Prompt:** The question or instruction provided to the model.
- **Token:** A small unit of text processed by the model.
- **Embedding:** A numerical vector representing a token.
- **Transformer:** The neural-network architecture used by most modern LLMs.
- **Self-Attention:** A mechanism that helps the model identify relationships within the context.
- **Parameters/Weights:** Numerical values learned during training.
- **Inference:** Using a trained model to generate a response.
- **Context Window:** The amount of information a model can process in a given interaction.
- **Hallucination:** A plausible-sounding but incorrect or unsupported generated statement.

When you ask an LLM a question, your prompt is first divided into tokens and converted into numerical representations. These representations pass through Transformer layers, where attention mechanisms help the model analyze relationships across the context.

The model then calculates which token should come next, selects one, adds it to the existing sequence, and repeats the process until the response is complete.

QUESTION

↓

TOKENS

↓

EMBEDDINGS

↓

TRANSFORMER

↓

ATTENTION

↓

NEXT-TOKEN PREDICTION

↓

TOKEN SELECTION

↓

REPEAT

↓

ANSWER

The key idea is:

**An LLM generates an answer by repeatedly predicting the next token based on the current context and the patterns encoded in its learned parameters.**